



💎💎 所属专栏: [Linux](#)💎💎

💎💎 作者主页: [数某](#)💎💎

[[Linux: 进程间通信]]

[[Linux: 进程-]]

[[Linux: 进程二-]]

[[Linux: ELF文件]]

[[Linux: 权限-]]

Linux: 进程信号

在讲信号之前，我们先来从生活中的事情来确定信号的一些特性。

- 我在网上买了商品，我在等快递。但是在快递没来之前我知道快递来的时候我应该怎么处理。我能**识别快递**。
- 快递来了，快递小哥给我打电话让我下楼取快递，而我在打游戏，等会再下去拿。在等会的这段时间里，我知道快递来了，但是我并没有立即去处理它，**没有立即执行**，也就是在合适的时候去取。
- 在这个等一会的时间窗口，我知道有一个快递在等我去取，那么我是记住了这件事的。
- 我拿到快递后有三种动作
 1. 执行默认动作（拆开快递）
 2. 自定义动作（送别人）
 3. 忽略（继续打游戏）
- 在整个来快递的过程中对我来说是**异步**的，我并不知道快递员什么时候给我打电话。

基本结论：

1. 我为什么能识别信号，信号是内置的，进程识别信号是内核程序员写的内置特性。
2. 信号产生后，我知道怎么处理。信号没有产生，我也知道怎么处理。所以信号的处理方法在信号产生之前，就已经准备好了。
3. 我们不一定立即处理信号，可能有优先级更高的事。那什么时候？合适的时候。
4. 三个步骤：信号到来 | 信号保存 | 信号处理

5. 怎么进行信号处理：默认、忽略、自定义，后续都叫做信号捕捉。

我们平时，`Ctrl + C` 其实就是在给前台进程发信号。键盘输入一个硬件中断，被OS获取，解释成信号，发送给目标前台进程。进程收到信号，引起进程退出。

系统函数

```
1 NAME
2     signal - ANSI C signal handling
3 SYNOPSIS
4     #include <signal.h>
5     typedef void (*sighandler_t)(int);
6     sighandler_t signal(int signum, sighandler_t handler);
7 参数说明：
8     signum: 信号编号
9     handler: 函数指针，表示更改信号的处理动作，当收到对应的信号，就回调执行handler方法
```

`ctrl + C`其实就是在给前台进程发送 `SIGINT` 即 2 号信号。

要注意的是，`signal` 函数仅仅是设置了特定信号的捕捉行为处理方式，并不是直接调用处理动作。如果后续特定信号没有产生，设置的捕捉函数永远也不会被调用！

`Ctrl + C` 产生的信号只能发给前台进程。一个命令后面加个 `&` 可以放到后台运行，这样 `Shell` 不必等待进程结束就可以接受新的命令，启动新的进程。

`Shell` 可以同时运行一个前台进程和任意多个后台进程，只有前台进程才能接收到像 `Ctrl + C` 这种控制键产生的信号。

前台进程在运行过程中用户随时可能按下 `Ctrl + C` 而产生一个信号，也就是说该进程用户空间代码执行到任何地方都有可能收到 `SIGINT` 信号而终止，所以信号相对于进程的控制流程来说是异步 `Asynchronous` 的

补充同步异步概念

一、同步 (Synchronous)

1. 定义

同步操作要求任务按顺序执行，前一个任务完成后才能启动下一个任务。主线程会**阻塞**等待当前任务返回结果，后续代码无法继续执行。

类比：类似排队办理银行业务，必须等待前一个人完成才能轮到下一个人。

2. 特点

- **顺序性：**代码执行顺序与编写顺序一致，逻辑简单。
- **阻塞性：**主线程在等待结果时会被挂起，可能导致界面卡顿或性能下降。

3. 应用场景

- 简单且非耗时操作（如变量赋值、数学计算）。
- 需要严格顺序执行的流程（如先登录后加载用户数据）。
- 要求数据强一致性的场景（如银行转账操作）。

二、异步 (Asynchronous)

1. 定义

异步操作将耗时任务放入后台执行，主线程**不等待结果**而继续执行后续代码。任务完成后通过回调函数、事件通知等方式返回结果。

类比：在餐厅点餐后领取号码牌，期间可自由活动，餐好后凭通知取餐。

2. 特点

- **非阻塞性：**主线程资源高效利用，避免卡顿。
- **复杂性：**需通过回调、Promise、async/await等机制处理结果。

3. 应用场景

- 耗时操作（如网络请求、文件读写）。
- 用户交互事件（如点击、滚动监听）。
- 高并发场景（如消息队列处理数据库批量写入）。

基础进程切换命令

1. & 符号

- **用途：**直接在命令末尾添加 `&`，使程序**立即在后台运行**
- 示例：

```
1 python script.py & # 脚本在后台运行
```

2. Ctrl+Z 组合键

- **用途：**暂停前台进程并将其转入后台（状态为 `Stopped`）
- **示例：**运行 `top` 时按下 `Ctrl+Z`，进程暂停并显示 `[1] (^) Stopped`

3. jobs 命令

- **用途：**查看当前Shell会话中的**后台任务列表**，显示任务编号和状态
- 常用参数

```
1 jobs -l # 显示任务PID
```

4. fg 命令

- **用途：**将后台任务**切换至前台继续运行**
- 语法：

```
1 fg %n # n为jobs显示的任务编号
```

- **示例：** `fg %1` 将编号1的任务调回前台

[58](#)

5. bg 命令

- **用途：**恢复暂停的**后台任务**，使其继续在后台运行
- 语法：

```
1 bg %n # 启动编号为n的暂停任务
```

- **示例：**暂停的 `top` 任务执行 `bg %1` 后转为后台运行

二、高级管理命令

1. `nohup` 命令

- **用途：**脱离终端运行程序，即使关闭SSH连接进程仍持续
- 示例：

```
1 nohup python server.py & # 输出日志到nohup.out
```

- **查看日志：** `tail -f nohup.out`

2. `kill` 命令

- **用途：**终止后台任务
- 语法：

```
1 kill %n          # 通过任务编号终止
2 kill <PID>       # 通过进程ID终止
```

- **示例：** `kill %2` 终止编号为2的任务

四、注意事项

- **任务编号与PID：** `fg/bg` 操作依赖 `jobs` 显示的**任务编号**，而非系统PID
- **终端依赖：**普通后台任务（未用 `nohup`）会随终端关闭终止
- **并发控制：**多个后台任务时，建议用 `jobs` 定期检查状态

信号概念

信号是进程之间事件异步通知的一种方式，属于软中断。

查看信号

```
1 ubuntu@VM-4-4-ubuntu:~/Code$ kill -l
2  1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
3  6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL    10) SIGUSR1
4 11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
5 16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
6 21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU    25) SIGXFSZ
7 26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO      30) SIGPWR
8 31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37)
  SIGRTMIN+3
9 38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42)
  SIGRTMIN+8
10 43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47)
  SIGRTMIN+13
11 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-
  12
12 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-
  7
```

```
13 58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-
2
14 63) SIGRTMAX-1 64) SIGRTMAX
```

每个进程都有一个编号和一个宏定义名称，这些宏定义可以在 `signal.h` 中找到。

```
1 #define SIGHUP          1 /* Hangup (POSIX). */
2 #define SIGINT          2 /* Interrupt (ANSI). */
3 #define SIGQUIT         3 /* Quit (POSIX). */
4 #define SIGILL          4 /* Illegal instruction (ANSI). */
5 #define SIGTRAP         5 /* Trace trap (POSIX). */
6 #define SIGIOT          6 /* IOT trap (4.2 BSD). */
7 #define SIGABRT         SIGIOT /* Abort (ANSI). */
8 #define SIGEMT          7
9 #define SIGFPE          8 /* Floating-point exception (ANSI). */
10 #define SIGKILL         9 /* kill, unblockable (POSIX). */
11 #define SIGBUS         10 /* BUS error (4.2 BSD). */
12 #define SIGSEGV        11 /* Segmentation violation (ANSI). */
13 #define SIGSYS         12
14 #define SIGPIPE        13 /* Broken pipe (POSIX). */
15 #define SIGALRM        14 /* Alarm clock (POSIX). */
16 #define SIGTERM        15 /* Termination (ANSI). */
17 #define SIGUSR1        16 /* User-defined signal 1 (POSIX). */
18 #define SIGUSR2        17 /* User-defined signal 2 (POSIX). */
19 #define SIGCHLD        18 /* Child status has changed (POSIX). */
20 #define SIGCLD         SIGCHLD /* Same as SIGCHLD (System V). */
21 #define SIGPWR         19 /* Power failure restart (System V). */
22 #define SIGWINCH       20 /* Window size change (4.3 BSD, Sun). */
23 #define SIGURG         21 /* Urgent condition on socket (4.2 BSD). */
24 #define SIGIO          22 /* I/O now possible (4.2 BSD). */
25 #define SIGPOLL        SIGIO /* Pollable event occurred (System V). */
26 #define SIGSTOP        23 /* Stop, unblockable (POSIX). */
27 #define SIGTSTP        24 /* Keyboard stop (POSIX). */
28 #define SIGCONT        25 /* Continue (POSIX). */
29 #define SIGTTIN        26 /* Background read from tty (POSIX). */
30 #define SIGTTOU        27 /* Background write to tty (POSIX). */
31 #define SIGVTALRM      28 /* Virtual alarm clock (4.2 BSD). */
32 #define SIGPROF        29 /* Profiling alarm clock (4.2 BSD). */
33 #define SIGXCPU        30 /* CPU limit exceeded (4.2 BSD). */
34 #define SIGXFSZ        31 /* File size limit exceeded (4.2 BSD). */
```

信号编号没有32、33所以只有62种信号，编号34以上的是实时信号，不讨论实时信号。这些信号各自在什么条件下产生，默认的处理动作是什么，在 `signal(7)` 中有详细说明： `man 7 signal`

```
1 standard signals
2     Linux supports the standard signals listed below. The second
   column of the table indicates which standard (if any) specified the signal:
   "P1990" indicates that the signal is described in the original
3     POSIX.1-1990 standard; "P2001" indicates that the signal was added in
   SUSv2 and POSIX.1-2001.
4     signal      Standard  Action    Comment
5
6     SIGABRT     P1990     Core      Abort signal from abort(3)
```

7	SIGALRM	P1990	Term	Timer signal from alarm(2)
8	SIGBUS	P2001	Core	Bus error (bad memory access)
9	SIGCHLD	P1990	Ign	Child stopped or terminated
10	SIGCLD	-	Ign	A synonym for SIGCHLD
11	SIGCONT	P1990	Cont	Continue if stopped
12	SIGEMT	-	Term	Emulator trap
13	SIGFPE	P1990	Core	Floating-point exception
14	SIGHUP	P1990	Term	Hangup detected on controlling
	terminal			
15				or death of controlling process
16	SIGILL	P1990	Core	Illegal Instruction
17	SIGINFO	-		A synonym for SIGPWR
18	SIGINT	P1990	Term	Interrupt from keyboard
19	SIGIO	-	Term	I/O now possible (4.2BSD)
20	SIGIOT	-	Core	IOT trap. A synonym for SIGABRT
21	SIGKILL	P1990	Term	Kill signal
22	SIGLOST	-	Term	File lock lost (unused)
23	SIGPIPE	P1990	Term	Broken pipe: write to pipe with no
24				readers; see pipe(7)
25	SIGPOLL	P2001	Term	Pollable event (Sys V);
26				synonym for SIGIO
27	SIGPROF	P2001	Term	Profiling timer expired
28	SIGPWR	-	Term	Power failure (System V)
29	SIGQUIT	P1990	Core	Quit from keyboard
30	SIGSEGV	P1990	Core	Invalid memory reference
31	SIGSTKFLT	-	Term	Stack fault on coprocessor (unused)
32	SIGSTOP	P1990	Stop	Stop process
33	SIGTSTP	P1990	Stop	Stop typed at terminal
34	SIGSYS	P2001	Core	Bad system call (SVr4);
35				see also seccomp(2)
36	SIGTERM	P1990	Term	Termination signal
37	SIGTRAP	P2001	Core	Trace/breakpoint trap
38	SIGTTIN	P1990	Stop	Terminal input for background process
39	SIGTTOU	P1990	Stop	Terminal output for background process
40	SIGUNUSED	-	Core	Synonymous with SIGSYS
41	SIGURG	P2001	Ign	Urgent condition on socket (4.2BSD)
42	SIGUSR1	P1990	Term	User-defined signal 1
43	SIGUSR2	P1990	Term	User-defined signal 2
44	SIGVTALRM	P2001	Term	Virtual alarm clock (4.2BSD)
45	SIGXCPU	P2001	Core	CPU time limit exceeded (4.2BSD);
46				see setrlimit(2)
47	SIGXFSZ	P2001	Core	File size limit exceeded (4.2BSD);
48				see setrlimit(2)
49	SIGWINCH	-	Ign	Window resize signal (4.3BSD, Sun)
50				
51	The signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.			

信号处理

- 忽略此信号

```

1 #include <iostream>
2 #include <unistd.h>
3 #include <signal.h>
4

```

```

5 void handler(int num)
6 {
7     std::cout << "我是: " << getpid() << ", 我获得了一个信号: " << signumber <<
std::endl;
8 }
9
10 int main()
11 {
12     std::cout << "我是进程: " << getpid() << std::endl;
13     signal(SIGINT /*2*/, SIG_IGN); // 设置忽略信号的宏
14     while (true)
15     {
16         std::cout << "I am a process, I am waiting signal!" << std::endl;
17         sleep(1);
18     }
19     return 0;
20 }
21 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ./sig
22 我是进程: 544272
23 I am a process, I am waiting signal!
24 I am a process, I am waiting signal!
25 I am a process, I am waiting signal!
26 I am a process, I am waiting signal!
27 I am a process, I am waiting signal!
28 ^CI am a process, I am waiting signal! // 输入 ctrl + C 毫无反应

```

- 执行该信号的默认处理动作

```

1 #include <iostream>
2 #include <unistd.h>
3 #include <signal.h>
4
5 void handler(int signumber)
6 {
7     std::cout << "我是: " << getpid() << ", 我获得了一个信号: " << signumber <<
std::endl;
8 }
9
10 int main()
11 {
12     std::cout << "我是进程: " << getpid() << std::endl;
13     signal(SIGINT /*2*/, SIG_DFL); // 设置默认处理
14     while (true)
15     {
16         std::cout << "I am a process, I am waiting signal!" << std::endl;
17         sleep(1);
18     }
19     return 0;
20 }
21
22 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ./sig
23 我是进程: 544934
24 I am a process, I am waiting signal!
25 I am a process, I am waiting signal!
26 I am a process, I am waiting signal!

```

```
27 ^C
28 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$
```

- 提供一个信号处理函数，要求内核在处理该信号是切换到用户态执行这个处理函数，这种方式称为自定义捕捉 `catch` 信号

```
1  #include <iostream>
2  #include <unistd.h>
3  #include <signal.h>
4
5  void handler(int signumber)
6  {
7      std::cout << "我是: " << getpid() << ", 我获得了一个信号: " << signumber <<
      std::endl;
8  }
9
10 int main()
11 {
12     std::cout << "我是进程: " << getpid() << std::endl;
13     signal(SIGINT /*2*/, handler); // 设置自定义函数处理
14     while (true)
15     {
16         std::cout << "I am a process, I am waiting signal!" << std::endl;
17         sleep(1);
18     }
19     return 0;
20 }
21
22 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ./sig
23 我是进程: 545560
24 I am a process, I am waiting signal!
25 I am a process, I am waiting signal!
26 I am a process, I am waiting signal!
27 ^C我是: 545560, 我获得了一个信号: 2
28 I am a process, I am waiting signal!
```

接下来我们将从产生信号，保存信号，捕捉信号三个方面来具体总结。

产生信号

通过终端按键产生信号

基本操作:

- `Ctrl + C` 向前台进程发送 `SIGINT` 信号
- `Ctrl + \` 发送终止信号 `SIGQUIT` 并生成core dump文件，用于事后调试。
- `Ctrl + Z` 发送停止信号 `SIGTSTP` 将当前前台进程挂起到后台等待。

ok这里就有一个问题了。键盘按下对应组合键是如何使进程进行对应操作的呢？键盘等硬件是直接或间接的与 CPU 上的针脚连接的，当按键按下，硬件发送一个中断给 CPU，CPU 识别到中断信息（高电平）然后就去执行处理硬件数据的代码。从操作系统来看就是 os 停下当前工作将数据从硬件读取到内存。

那么，信号就是从纯软件的角度来模拟硬件中断。硬件中断是发给 CPU 软中断是发给进程。两者在思想上是完全一致的。

使用函数产生信号

kill

我们在终端使用的 `kill` 命令本质也是进程，也是用C语言写的。底层也是调用的这个 `kill` 函数。`kill` 函数会给一个指定的进程发送指定的信号

```
1 NAME
2     kill - send signal to a process
3 SYNOPSIS
4     #include <sys/types.h>
5     #include <signal.h>
6     int kill(pid_t pid, int sig);
7 RETURN VALUE
8     On success (at least one signal was sent) zero is returned. On error, -1
9     is returned, and errno is set
    appropriately.
```

mykill

```
1 #include <iostream>
2 #include <unistd.h>
3 #include <signal.h>
4 #include <sys/types.h>
5
6 // 实现自己的kill命令
7 // mykill -signumber pid
8 int main(int argc, char *argv[])
9 {
10     if (argc != 3)
11     {
12         std::cerr << "Usage: " << argv[0] << " -signumber pid" << std::endl;
13         return 1;
14     }
15     int number = std::stoi(argv[1] + 1); // 去掉- 获取信号编号
16     pid_t pid = std::stoi(argv[2]);
17     int n = kill(pid, number);
18     return n;
19 }
```

raise

`raise` 函数可以给当前进程发送指定的信号，也就是给自己发信号。

```

1 NAME
2     raise - send a signal to the caller
3 SYNOPSIS
4     #include <signal.h>
5     int raise(int sig);
6 RETURN VALUE
7     raise() returns 0 on success, and nonzero for failure.

```

abort

`abort` 函数使当前进程收到信号而异常终止，它总会成功的就像 `exit` 一样

```

1 NAME
2     abort - cause abnormal process termination
3 SYNOPSIS
4     #include <stdlib.h>
5     void abort(void);
6 RETURN VALUE
7     The abort() function never returns.

```

由软件产生信号

`SIGPIPE` 和 `SIGALRM` 信号是一种由软件产生的信号，管道我们已经学过了。现在来学习时钟信号

`alarm` 函数

```

1 NAME
2     alarm - set an alarm clock for delivery of a signal
3 SYNOPSIS
4     #include <unistd.h>
5     unsigned int alarm(unsigned int seconds);
6 DESCRIPTION
7     alarm() arranges for a SIGALRM signal to be delivered to the calling
    process in seconds seconds.
8     If seconds is zero, any pending alarm is canceled.
9     In any event any previously set alarm() is canceled.
10 RETURN VALUE
11     alarm() returns the number of seconds remaining until any previously
12     scheduled alarm was due to be delivered, or zero if there was no previ-
13     ously scheduled alarm.

```

- 调用 `alarm` 函数可以设定一个闹钟，也就是告诉内核在 `seconds` 秒之后给当前进程发 `SIGALRM` 信号，该信号的默认处理动作是终止当前进程。
- 这个函数的返回值是0或者是以前设定的闹钟时间还余下的秒数。如果 `seconds` 的值为0，表示取消以前设定的闹钟，函数的返回值仍然是以前设定的闹钟时间还余下的秒数。

IO效率问题

这里延申出一个IO效率的问题，分别有两个程序，一个程序在一秒的时间内不断向显示屏打印信息并使计数器加加，另一个在一秒钟之内不断只对一个计数器加加。最后看它们的计数器大小。

```

1 #include <iostream>
2 #include <unistd.h>

```

```

3  #include <signal.h>
4  #include <sys/types.h>
5  int cnt = 0;
6
7  void Exit(int signo)
8  {
9      std::cout << cnt << std::endl;
10     exit(1);
11 }
12
13 int main()
14 {
15     signal(SIGALRM, Exit);
16     alarm(1);
17
18     while (true)
19     {
20         cnt++;
21     }
22     return 0;
23 }
24
25 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ make
26 g++ -o sig sig.cc -std=c++11
27 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ./sig
28 560154357

```

```

1  #include <iostream>
2  #include <unistd.h>
3  #include <signal.h>
4  #include <sys/types.h>
5  int cnt = 0;
6
7  void Exit(int signo)
8  {
9      std::cout << cnt << std::endl;
10     exit(1);
11 }
12
13 int main()
14 {
15     signal(SIGALRM, Exit);
16     alarm(1);
17
18     while (true)
19     {
20         std::cout << "cnt:" << cnt << std::endl;
21         cnt++;
22     }
23     return 0;
24 }
25
26 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ make
27 g++ -o sig sig.cc -std=c++11
28 ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ./sig

```

```

29 cnt:0
30 cnt:1
31 cnt:2
32 .....
33 cnt:53558
34 cnt:53559
35 cnt:53559

```

事实证明，相比于这种算术运算，IO处理的是非常慢的。两者之间不止差了一个数量级。

设置重复闹钟

```

1  #include <iostream>
2  #include <vector>
3  #include <functional>
4  #include <unistd.h>
5  #include <signal.h>
6  #include <sys/types.h>
7
8  using func_t = std::function<void()>;
9  std::vector<func_t> task;
10
11 void handler(int signo)
12 {
13     for (auto f : task)
14     {
15         f();
16     }
17     int n = alarm(1);
18     std::cout << "n: " << n << std::endl;
19 }
20
21 int main()
22 {
23     task.push_back([]()
24         { std::cout << "刷新内核" << std::endl; });
25     task.push_back([]()
26         { std::cout << "检测进程时间片" << std::endl; });
27     task.push_back([]()
28         { std::cout << "管理内存" << std::endl; });
29
30     signal(SIGALRM, handler);
31
32     alarm(1);
33
34     while (true)
35     {
36         pause();
37         std::cout << "wake up" << std::endl;
38     }
39     return 0;
40 }
41 /*****
42     *****/
43 NAME

```

```

43     pause - wait for signal
44
45 SYNOPSIS
46     #include <unistd.h>
47     int pause(void);
48
49 DESCRIPTION
50     pause() causes the calling process (or thread) to sleep until a
    signal is delivered that either terminates the process or causes the
    invocation of a signal-catching function.
51
52 RETURN VALUE
53     pause() returns only when a signal was caught and the signal-catching
    function returned. In this case, pause() returns -1, and errno is set to
    EINTR.

```

硬件异常产生信号

硬件异常被硬件以某种方式被硬件检测到并通知内核，然后内核发送适当的信号给当前进程。例如当前进程进行了除0的指令，CPU的运算单位会产生异常，内核将这个异常解释成 SIGFPE 信号发给进程。如果当前进程访问非法内存地址，MMU 会产生异常，内核将在这个异常解释为 SIGSEGV 信号发送给进程。

core dump

在一些信号的默认 action 中（上翻 查看信号）有 ign、core 等。它们是什么意思？

- SIGINT 的默认处理动作是终止进程，SIGQUIT 的默认处理动作是终止进程并且 Core Dump，当一个进程要异常终止时，可以把进程的用户空间内存数据全部保存到磁盘上，文件名通常是 core，这叫做 Core Dump。
- 进程异常终止通常是因为有 Bug 比如非法内存访问导致段错误，事后可以用调试器检查 core 文件以查清错误原因，这叫做 Post-mortem DeBug 事后调试。
- 一个进程允许产生多大的 core 文件取决于进程的 Resource Limit（在PCB中），默认不允许产生 core 文件，因为可能包含用户密码等敏感信息。
- 在开发测试阶段可以用 ulimit 命令该变这个限制，允许产生 core 文件。先用这个命令改变 Shell 进程的 Resource Limit，如改到 1024k: \$ ulimit -c 1024

```

1  ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ulimit -c 1024
2  ubuntu@VM-4-4-ubuntu:~/Code/25/3_15$ ulimit -a
3  real-time non-blocking time (microseconds, -R) unlimited
4  core file size              (blocks, -c) 1024
5  data seg size               (kbytes, -d) unlimited
6  scheduling priority         (-e) 0
7  file size                   (blocks, -f) unlimited
8  pending signals             (-i) 6553
9  max locked memory           (kbytes, -l) 219108
10 max memory size             (kbytes, -m) unlimited
11 open files                  (-n) 1048576
12 pipe size                   (512 bytes, -p) 8
13 POSIX message queues        (bytes, -q) 819200
14 real-time priority          (-r) 0
15 stack size                  (kbytes, -s) 8192
16 cpu time                    (seconds, -t) unlimited

```

```

17 max user processes          (-u) 6563
18 virtual memory              (kbytes, -v) unlimited
19 file locks                   (-x) unlimited

```

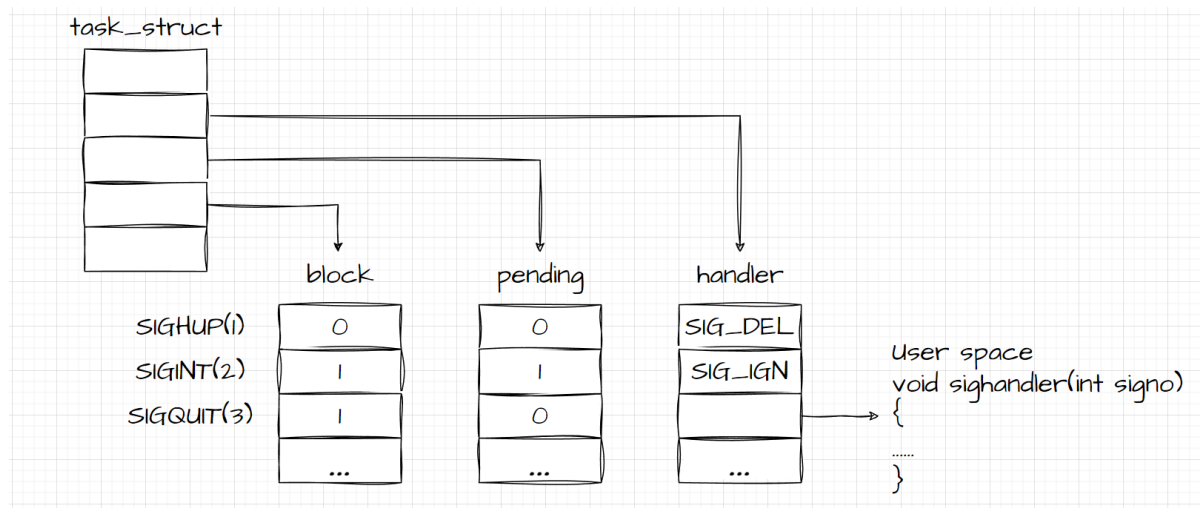
保存信号

上面我们说了信号产生后都需要OS来执行，因为OS是进程的管理者。但信号不是被立即处理的，是在合适的时候处理。那既然不是立即处理，总该保存吧，那保存在哪里呢？

概念说明：

- 实际执行信号处理动作称为信号递达 **Delivery**
- 信号从产生到递达之间的状态成为信号未决 **Pending**
- 进程可以选择阻塞 **Block** 某个信号
- **被阻塞的信号产生时将保持在未决状态，直到进程解除对此信号的阻塞，才执行递达的动作**
- 注意：阻塞和忽略是不同的，忽略是在信号递达后可选的一种处理方式，但信号被阻塞了就无法递达

在内核中进程 PCB 维护了这三张表：



- 每个信号都有两个标志位分别表示阻塞 **block** 和未决 **pending**，还有一个函数指针表示处理动作。信号产生时，内核在进程控制块中设置该信号的未决标志，直到信号递达才清除标志。在上图例子中，**SIGHUP** 信号未阻塞也未产生过，当它递达时执行默认处理动作。
- **SIGINT** 信号产生过，但正在被阻塞，所以暂时不能被递达，虽然它的处理动作为忽略，但在没有解除阻塞之前不能忽略这个信号，因为进程仍有机会修改 **handler** 并解除忽略。
- **SIGQUIT** 信号未产生过，一旦产生 **SIGQUIT** 信号将被阻塞，它的处理动作时用户自定义函数 **sighandler**。

如果在进程解除对某信号的阻塞之前这种信号产生过多次,将如何处理？

POSIX.1允许系统递送该信号一次或多次。Linux是这样实现的：常规信号在递达之前产生多次只计一次,而实时信号在递达之前产生多次可以依次放在一个队列里。我们暂时不讨论实时信号。

```

1 // 内核结构
2 struct task_struct
3 {
4     ...
5     /* signal handlers */

```

```

6     struct sighand_struct *sighand;
7     sigset_t blocked struct sigpending pending;
8     ...
9
10 }
11
12 struct sighand_struct
13 {
14     atomic_t count;
15     struct k_sigaction action[_NSIG]; // #define _NSIG 64
16     spinlock_t siglock;
17 };
18
19 struct __new_sigaction
20 {
21     __sighandler_t sa_handler;
22     unsigned long sa_flags;
23     void (*sa_restorer)(void); /* Not used by Linux/SPARC */
24     __new_sigset_t sa_mask;
25 };
26
27 struct k_sigaction
28 {
29     struct __new_sigaction sa;
30     void __user *ka_restorer;
31 };
32
33 /* Type of a signal handler. */
34 typedef void (*__sighandler_t)(int);
35 struct sigpending
36 {
37     struct list_head list;
38     sigset_t signal;
39 };

```

sigset_t

从上图来看，每个信号只有一个bit的未决标志，非0即1，不记录该信号产生了多少次，阻塞标志也是这样表示的。因此，未决和阻塞标志可以用相同的数据类型 `sigset_t` 来存储，`sigset_t` 称为信号集，这个类型可以表示每个信号的“有效”或“无效”状态，在阻塞信号集中“有效”和“无效”的含义是该信号是否被阻塞，而在未决信号集中“有效”和“无效”的含义是该信号是否处于未决状态。阻塞信号集也叫做当前进程的信号屏蔽字 `Signal Mask` 这里的“屏蔽”应该理解为阻塞而不是忽略。

信号集操作函数

`sigset_t` 类型对于每种信号用一个 `bit` 表示“有效”或“无效”状态，至于这个类型内部如何存储这些 `bit` 则依赖于系统实现，从使用者的角度是不必关心的，使用者只能调用以下函数来操作 `sigset_t` 变量，而不应该对它的内部数据做任何解释，**比如用 `printf` 直接打印 `sigset_t` 变量是没有意义的。**

```

1 #include <signal.h>
2 int sigemptyset(sigset_t *set);
3 int sigfillset(sigset_t *set);
4 int sigaddset(sigset_t *set, int signo);
5 int sigdelset(sigset_t *set, int signo);
6 int sigismember(const sigset_t *set, int signo);

```

- 函数 `sigemptyset` 初始化 `set` 所指向的信号集，使其中所有信号的对应 bit 清零，表示该信号集不包含任何有效信号。
- 函数 `sigfillset` 初始化 `set` 所指向的信号集，使其中所有信号的对应 bit 置一，表示该信号集包含所有可能的有效信号，包括系统支持的所有信号。
- 注意,在使用 `sigset_t` 类型的变量之前，一定要调用 `sigemptyset` 或 `sigfillset` 做初始化，使信号集处于确定的状态。初始化 `sigset_t` 变量之后就可以在调用 `sigaddset` 和 `sigdelset` 在该信号集中添加或删除某种有效信号。

这四个函数都是成功返回0，出错返回-1。`sigismember` 是一个布尔函数，用于判断一个信号集的有效信号中是否包含某种信号，若包含则返回1，不包含则返回0，出错返回-1。

sigprocmask

函数 `sigprocmask` 可以读取或更改进程的信号屏蔽字 `block`

```

1 #include <signal.h>
2 int sigprocmask(int how, const sigset_t *set, sigset_t *oset);
3 // 返回值：成功返回0，出错返回-1

```

如果 `oset` 指针非空，则读取进程的当前信号屏蔽字通过 `oset` 传出。如果 `set` 非空，则更改进程的信号屏蔽字，参数 `how` 指示如何修改。如果 `oset` 和 `set` 均非空，则将进程的原来的信号屏蔽字备份到 `oset` 并根据 `set` 和 `how` 修改信号屏蔽字。假设当前信号屏蔽字为 `mask` 下标说明了 `how` 参数的可选值

SIG_BLOCK	<code>set</code> 包含了我们希望添加到当前信号屏蔽字的信号，相当于 <code>mask = mask set</code>
SIG_UNBLOCK	<code>set</code> 包含了我们希望从当前信号屏蔽字种解除阻塞的信号，相当于 <code>mask = mask & ~set</code>
SIG_SETMASK	设置当前信号屏蔽字为 <code>set</code> 指向的值，相当于 <code>mask = set</code>

POSIX 标准规定，在 `sigprocmask` 返回前，内核必须确保至少有一个未决信号被递达。换句话说，解除阻塞后，内核会立即检查未决信号，并处理至少一个。

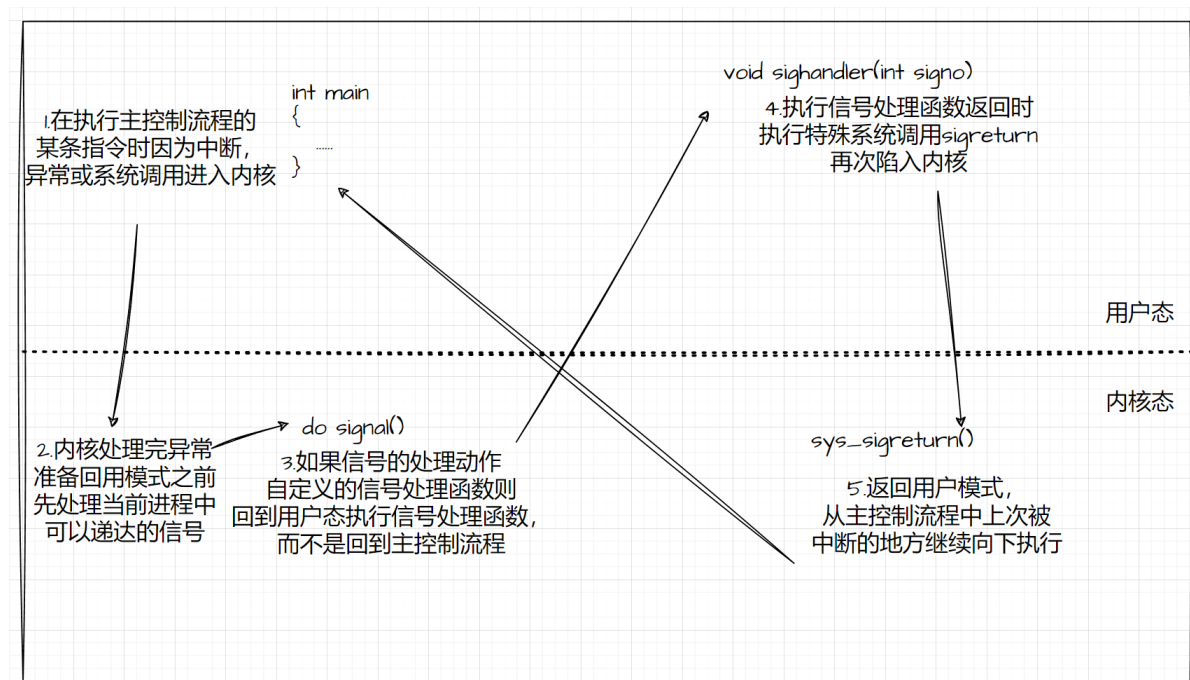
- **及时性**：确保信号不会被无限期延迟。如果解除阻塞后不立即递达，未决信号可能被后续的信号掩码操作重新阻塞。
- **原子性**：`sigprocmask` 的调用是原子的，即解除阻塞和递达信号的过程不可被中断。

sigpending

```
1 #include <signal.h>
2 int sigpending(sigset_t *set);
3
4 读取当前进程的未决信号集，通过set传出
5 调用成功返回0，出错返回-1
```

捕捉信号

信号捕捉的流程

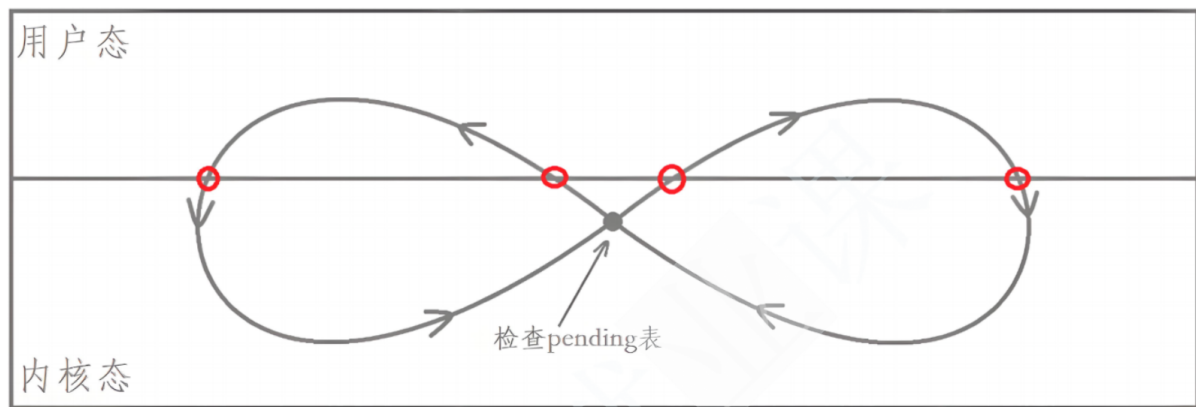


如果信号的处理动作是用户的自定义函数在信号递达时就调用这个函数，这个叫做捕捉信号。

处理信号的代码在用户空间，而信号的保存是在内核里面，所以这里就涉及到内核与用户的转换了

- 用户程序注册了 `SIG_INT` 信号的处理函数 `sighandler`。
- 当前执行 `main` 函数，在程序收到信号时，发生中断，陷入内核，中断处理完后发现 `pending` 表里面有信号 `SIG_INT` 递达。
- 内核返回用户态，但不是恢复 `main` 函数的上下文继续执行，而是执行 `sighandler` 函数，`sighandler` 和 `main` 函数在不同的堆栈空间，不存在调用或被调用关系，是两个独立的控制流。
- `sighandler` 函数返回后自动执行特殊的系统调用 `sigreturn` 再次进入内核态。
- 如果没有新的信号要递达，这次再返回用户态就是恢复 `main` 函数的上下文执行了。

我们把这个过程图抽象一下，就是数学里的无穷符号



sigaction

```
1 #include <signal.h>
2 int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);
```

- `sigaction` 函数可以读取和修改与指定信号相关的处理动作。调用成功则返回0，出错则返回-1。 `signo` 指信号编号。若 `act` 非空，则根据 `act` 修改该信号的编号，若 `oact` 非空则通过 `oact` 传出该信号原来的处理动作。 `act` 和 `oact` 都指向 `sigaction` 结构体：

```
1 // linux kernel 2.6.18
2 struct sigaction {
3     __sighandler_t sa_handler;
4     unsigned long sa_flags;
5     __sigrestore_t sa_restorer;
6     sigset_t sa_mask;      /* mask last for extensibility */
7 };
```

- 将 `sa_handler` 赋值为常数 `SIG_IGN` 传给 `sigaction` 表示忽略信号，赋值为 `SIG_DFL` 表示为执行系统默认动作，赋值为一个函数指针表明用自定义函数捕捉信号，或者说向内核注册了一个信号处理函数，该函数返回值为 `void`，一个 `int` 参数表示信号编号。显然这也是回调，不是被 `main` 回调，而是操作系统。

当某个信号的处理函数被调用时，进程会自动将当前信号加入当前进程的信号屏蔽字里，当前信号处理函数返回时自动恢复到原来的信号屏蔽字。确保在处理某个信号时，如果这种信号再次产生，那么它就会被阻塞到这个信号处理完为止。如果在调用信号处理函数时。除了当前信号被自动屏蔽外，还希望系统屏蔽另外一些信号，则用 `sa_mask` 字段说明这些需要额外屏蔽的信号，当信号处理函数返回时自动恢复原来的信号屏蔽字。 `sa_flag` 字段包含一些选项，其作用是通过不同的标志位来指定信号处理的一些特殊行为。

SA_NOCLDWAIT

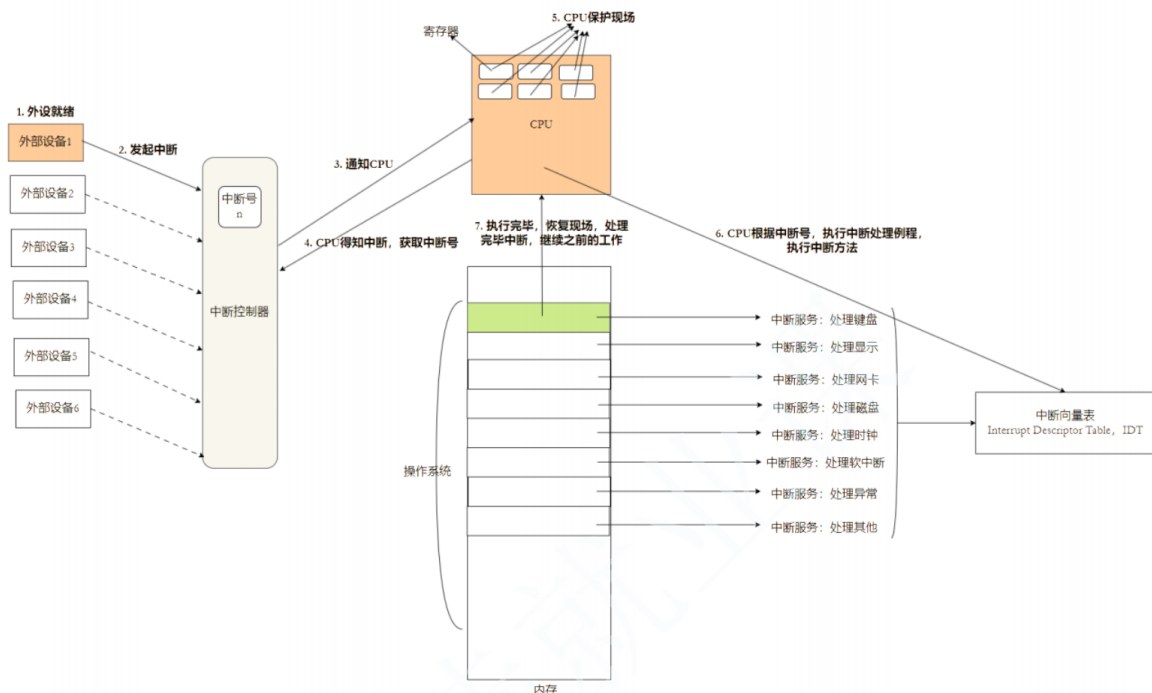
若信号为 `SIGCHLD`，同时设置了 `SA_NOCLDWAIT` 标志，在子进程终止时，系统不会将其转变为僵尸进程。这意味着父进程无需调用 `wait()` 或者 `waitpid()` 来回收子进程的资源。

SA_NODEFER

此标志表明在信号处理函数执行期间，不会自动阻塞当前正在处理的信号。一般而言，当进程正在处理某个信号时，内核会自动阻塞该信号，防止信号的嵌套处理。若设置了 `SA_NODEFER` 标志，那么在信号处理函数执行期间，相同的信号仍能被接收和处理。

操作系统是怎么运行的

硬件中断



中断向量表是操作系统的一部分，启动后自动加载到内存。

通过外部中断，操作系统就不需要对外设进行任何周期性的检测轮询。外部设备准备好了就发送中断，CPU收到中断号，对应去执行中断向量表里面的方法。等等，这一幕似曾相识，外设发送的中断好像信号啊，中断向量表也有点类似于 `handler` 表。这里面的思想都是一样的

由外部系统触发的，中断系统运行流程，叫做硬件中断。

```
1 // linux kernel 1.0
2 void trap_init(void)
3 {
4     int i;
5
6     set_trap_gate(0,&divide_error);// 设置除操作出错的中断向量值。以下雷同。
7     set_trap_gate(1,&debug);
8     set_trap_gate(2,&nmi);
9     set_system_gate(3,&int3); /* int3-5 can be called from all */
10    set_system_gate(4,&overflow);
11    set_system_gate(5,&bounds);
12    set_trap_gate(6,&invalid_op);
13    set_trap_gate(7,&device_not_available);
14    set_trap_gate(8,&double_fault);
15    set_trap_gate(9,&coprocessor_segment_overrun);
16    set_trap_gate(10,&invalid_TSS);
17    set_trap_gate(11,&segment_not_present);
18    set_trap_gate(12,&stack_segment);
19    set_trap_gate(13,&general_protection);
20    set_trap_gate(14,&page_fault);
21    set_trap_gate(15,&reserved);
22    set_trap_gate(16,&coprocessor_error);// 下面将int17-48 的陷阱门先均设置为
    reserved, 以后每个硬件初始化时会重新设置自己的陷阱门。
23    set_trap_gate(17,&alignment_check);
```

```

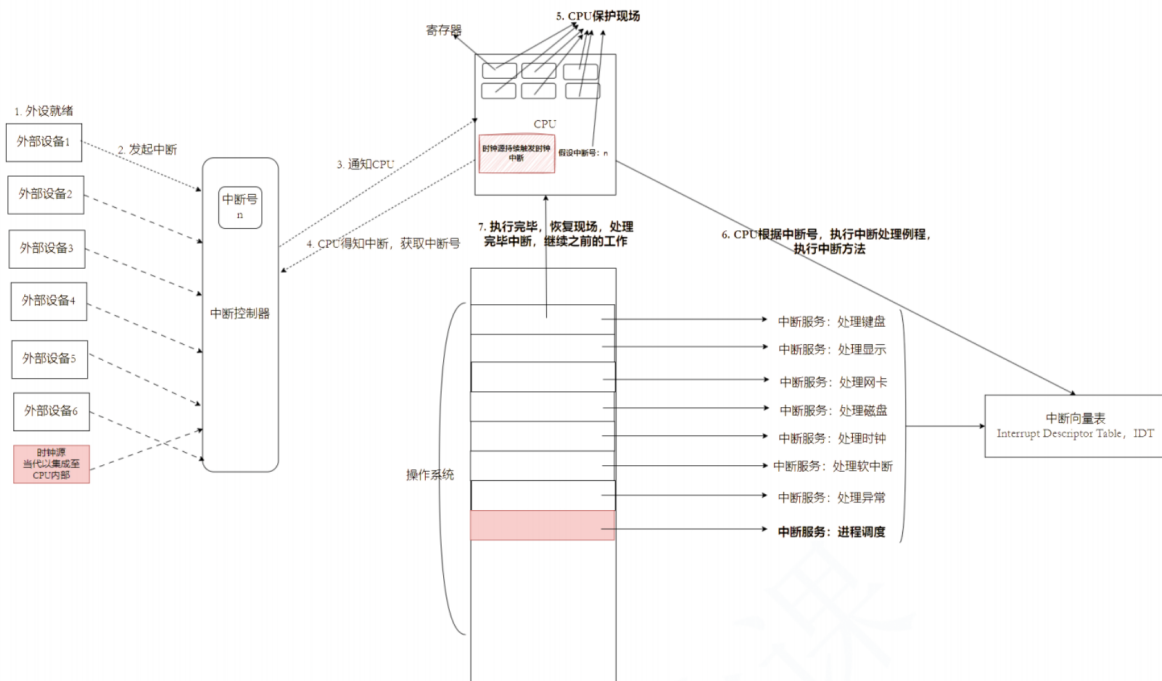
24     for (i=18;i<48;i++)
25         set_trap_gate(i,&reserved);
26 }

```

时钟中断

进程可以在操作系统的指挥下被调度被执行，那操作系统自己被谁推动执行？外设虽然可以自己或通过用户发送中断，那有没有可以自己定时发送中断的设备呢？有的兄弟有的！！

时钟源现在已经被集成在了 CPU 内部，我们通常说，CPU 的主频是多少，在某种程度上就是 CPU 内部的时钟源的频率。在架构和核心数量相同的情况下，主频越高，CPU 每秒钟能调度的进程就越多。（时钟源频率和主频并不相等，中间有转换公式 $\text{主频} = \text{时钟源频率} \times \text{倍频系数}$ ）



现在操作系统什么都不需要做，在时钟中断的推动下，自动调度进程。

```

1 // Linux 内核0.11
2 // main.c
3 sched_init(); // 调度程序初始化(加载了任务0 的tr, ldtr) (kernel/sched.c)
4
5 // 调度程序的初始化子程序。
6 void sched_init(void)
7 {
8     ...
9     set_intr_gate(0x20, &timer_interrupt);
10    // 修改中断控制器屏蔽码, 允许时钟中断。
11    outb(inb_p(0x21) & ~0x01, 0x21);
12    // 设置系统调用中断门。
13    set_system_gate(0x80, &system_call);
14    ...
15 }
16
17 // system_call.s
18 _timer_interrupt :
19     ...
20 ; // do_timer(CPL)执行任务切换、计时等工作, 在kernel/shched.c, 305 行实现。
21 call _do_timer; // 'do_timer(long CPL)' does everything from

```

```

22
23 // 调度入口
24 void do_timer(long cpl)
25 {
26     ...
27     schedule();
28 }
29 void schedule(void)
30 {
31     ...
32     switch_to(next); // 切换到任务号为next 的任务，并运行之。
33 }
34

```

死循环

如果有了时钟中断，操作系统不就可以什么都不做了吗，需要什么功能就向中断向量表里加方法。操作系统的本质就是死循环。

```

1 // linux kernel 0.11
2 void main(void) /* 这里确实是void，并没错。 */
3 {
4     /*
5      * 注意!! 对于任何其它的任务，'pause()'将意味着我们必须等待收到一个信号才会返
6      * 回就绪运行态，但任务0 (task0) 是唯一的意外情况（参见'schedule()'），因为任
7      * 务0 在任何空闲时间里都会被激活（当没有其它任务在运行时），
8      * 因此对于任务0 'pause()' 仅意味着我们返回来查看是否有其它任务可以运行，如果没
9      * 有的话我们就回到这里，一直循环执行'pause()'。
10     */
11     for (;;)
12         pause();
13 } // end main
14
15
16 // linux kernel 1.0
17 asmlinkage void start_kernel(void)
18 {
19     ...
20     trap_init();
21     init_IRQ();
22     sched_init();
23     buffer_init();
24     time_init();
25     floppy_init();
26     sock_init();
27     ...
28     /*
29      * task[0] is meant to be used as an "idle" task: it may not sleep, but
30      * it might do some general things like count free pages or it could be
31      * used to implement a reasonable LRU algorithm for the paging routines:
32      * anything that can be useful, but shouldn't take time from the real
33      * processes.
34      *
35      * Right now task[0] just does a infinite idle loop.
36      */

```

```

37     for(;;)
38         idle();
39 }

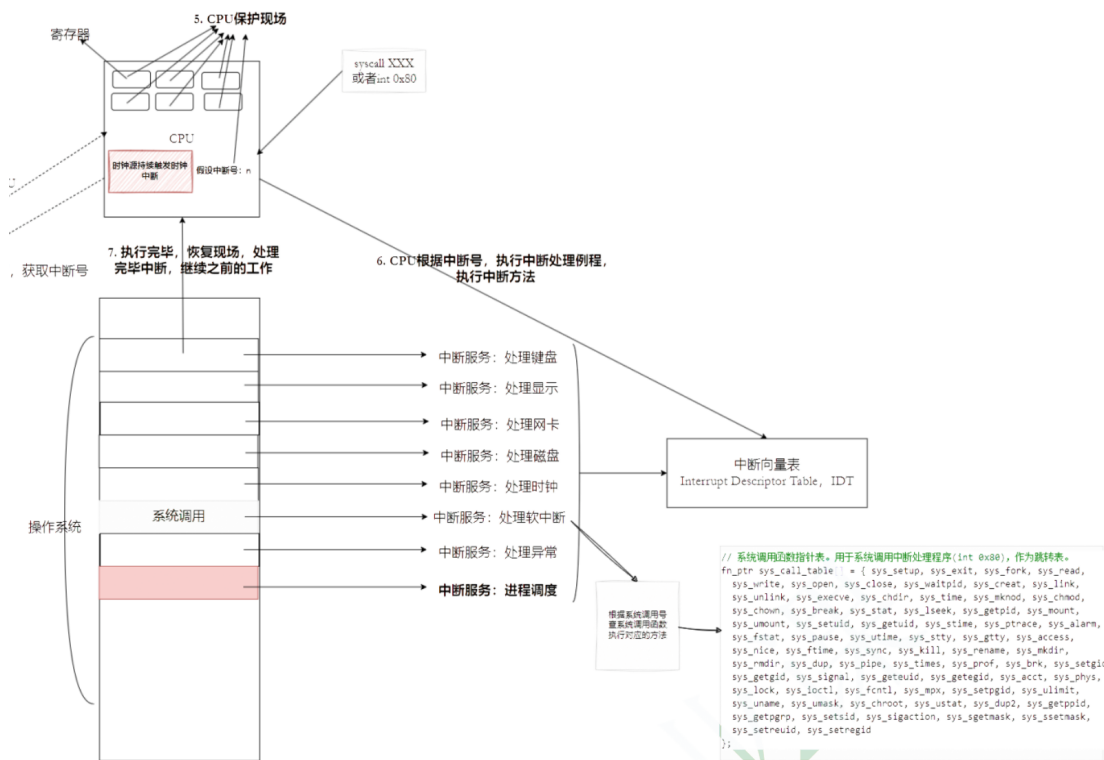
```

所以，CPU 每隔一段时间（时钟中断频率）对当前进程的时间片减减，减到零，将当前进程放入等待队列，从执行队列里拿一个进程过来执行。这不就是进程调度吗~。所以为什么主频越快，CPU 越快，也是因为这个。

软中断

上述的中断都是由硬件触发，那么有没有由软件触发的中断呢？有的兄弟有的！！

为了让操作系统支持进行系统调用，CPU 也设计了对应的汇编指令（`int` 或者 `syscall`）可以让 CPU 内部触发中断逻辑，只需要再在中断向量表中加方法就行啦。



用户通过寄存器将系统调用号传给操作系统，操作系统也通过寄存器或用户提供的缓冲区地址返回。

系统调用的过程就是，操作系统先 `int 0x80` 或者 `syscall` 陷入内核，本质就是触发软中断，CPU 自动执行系统调用的处理方法。而这个方法会根据系统调用号，自动查表，执行方法。系统调用号的本质就是数组下标。

```

1 // linux kernel 1.0
2 // sys.h
3 extern int sys_setup();          /* 0 */
4 extern int sys_exit();
5 extern int sys_fork();
6 extern int sys_read();
7 extern int sys_write();
8 extern int sys_open();          /* 5 */
9 extern int sys_close();
10 extern int sys_waitpid();
11 extern int sys_creat();
12 extern int sys_link();

```

```
13 extern int sys_unlink();          /* 10 */
14 extern int sys_execve();
15 extern int sys_chdir();
16 extern int sys_time();
17 extern int sys_mknod();
18 extern int sys_chmod();          /* 15 */
19 extern int sys_chown();
20 extern int sys_break();
21 extern int sys_stat();
22 extern int sys_lseek();
23 extern int sys_getpid();        /* 20 */
24 extern int sys_mount();
25 extern int sys_umount();
26 extern int sys_setuid();
27 extern int sys_getuid();
28 extern int sys_stime();         /* 25 */
29 extern int sys_ptrace();
30 extern int sys_alarm();
31 extern int sys_fstat();
32 extern int sys_pause();
33 extern int sys_utime();         /* 30 */
34 extern int sys_stty();
35 extern int sys_gtty();
36 extern int sys_access();
37 extern int sys_nice();
38 extern int sys_ftime();         /* 35 */
39 extern int sys_sync();
40 extern int sys_kill();
41 extern int sys_rename();
42 extern int sys_mkdir();
43 extern int sys_rmdir();        /* 40 */
44 extern int sys_dup();
45 extern int sys_pipe();
46 extern int sys_times();
47 extern int sys_prof();
48 extern int sys_brk();          /* 45 */
49 extern int sys_setgid();
50 extern int sys_getgid();
51 extern int sys_signal();
52 extern int sys_geteuid();
53 extern int sys_getegid();      /* 50 */
54 extern int sys_acct();
55 extern int sys_phys();
56 extern int sys_lock();
57 extern int sys_ioctl();
58 extern int sys_fcntl();        /* 55 */
59 extern int sys_mpx();
60 extern int sys_setpgid();
61 extern int sys_ulimit();
62 extern int sys_uname();
63 extern int sys_umask();        /* 60 */
64 extern int sys_chroot();
65 extern int sys_ustat();
66 extern int sys_dup2();
67 extern int sys_getppid();
68 extern int sys_getpgrp();      /* 65 */
```

```
69 extern int sys_setsid();
70 extern int sys_sigaction();
71 extern int sys_sgetmask();
72 extern int sys_ssetmask();
73 extern int sys_setreuid(); /* 70 */
74 extern int sys_setregid();
75 extern int sys_sigpending();
76 extern int sys_sigsuspend();
77 extern int sys_sethostname();
78 extern int sys_setrlimit(); /* 75 */
79 extern int sys_getrlimit();
80 extern int sys_getrusage();
81 extern int sys_gettimeofday();
82 extern int sys_settimeofday();
83 extern int sys_getgroups(); /* 80 */
84 extern int sys_setgroups();
85 extern int sys_select();
86 extern int sys_symlink();
87 extern int sys_lstat();
88 extern int sys_readlink(); /* 85 */
89 extern int sys_uselib();
90 extern int sys_swapon();
91 extern int sys_reboot();
92 extern int sys_readdir();
93 extern int sys_mmap(); /* 90 */
94 extern int sys_munmap();
95 extern int sys_truncate();
96 extern int sys_ftruncate();
97 extern int sys_fchmod();
98 extern int sys_fchown(); /* 95 */
99 extern int sys_getpriority();
100 extern int sys_setpriority();
101 extern int sys_profil();
102 extern int sys_statfs();
103 extern int sys_fstatfs(); /* 100 */
104 extern int sys_ioperm();
105 extern int sys_socketcall();
106 extern int sys_syslog();
107 extern int sys_getitimer();
108 extern int sys_setitimer(); /* 105 */
109 extern int sys_newstat();
110 extern int sys_newlstat();
111 extern int sys_newfstat();
112 extern int sys_newuname();
113 extern int sys_iopl(); /* 110 */
114 extern int sys_vhangup();
115 extern int sys_idle();
116 extern int sys_vm86();
117 extern int sys_wait4();
118 extern int sys_swapoff(); /* 115 */
119 extern int sys_sysinfo();
120 extern int sys_ipc();
121 extern int sys_fsync();
122 extern int sys_sigreturn();
123 extern int sys_setdomainname(); /* 120 */
124 extern int sys_olduname();
```



```

125 extern int sys_old_syscall();
126 extern int sys_modify_ldt();
127 extern int sys_adjtimex();
128 extern int sys_mprotect(); /* 125 */
129 extern int sys_sigprocmask();
130 extern int sys_create_module();
131 extern int sys_init_module();
132 extern int sys_delete_module();
133 extern int sys_get_kernel_syms(); /* 130 */
134 extern int sys_quotactl();
135 extern int sys_getpgid();
136 extern int sys_fchdir();
137 extern int sys_bdflush();
138
139 /*
140  * These are system calls that will be removed at some time
141  * due to newer versions existing..
142  */
143
144 // 系统调用函数指针表。用于系统调用中断处理程序(int 0x80)，作为跳转表。
145 fn_ptr sys_call_table[] = { sys_setup, sys_exit, sys_fork, sys_read,
146     sys_write, sys_open, sys_close, sys_waitpid, sys_creat, sys_link,
147     sys_unlink, sys_execve, sys_chdir, sys_time, sys_mknod, sys_chmod,
148     sys_chown, sys_break, sys_stat, sys_lseek, sys_getpid, sys_mount,
149     sys_umount, sys_setuid, sys_getuid, sys_stime, sys_ptrace, sys_alarm,
150     sys_fstat, sys_pause, sys_utime, sys_stty, sys_gtty, sys_access,
151     sys_nice, sys_ftime, sys_sync, sys_kill, sys_rename, sys_mkdir,
152     sys_rmdir, sys_dup, sys_pipe, sys_times, sys_prof, sys_brk, sys_setgid,
153     sys_getgid, sys_signal, sys_geteuid, sys_getegid, sys_acct, sys_phys,
154     sys_lock, sys_ioctl, sys_fcntl, sys_mpx, sys_setpgid, sys_ulimit,
155     sys_uname, sys_umask, sys_chroot, sys_ustat, sys_dup2, sys_getppid,
156     sys_getpgrp, sys_setsid, sys_sigaction, sys_sgetmask, sys_ssetmask,
157     sys_setreuid, sys_setregid
158 };

```

- 我们使用的系统调用本质是由 `c` 标准库 对下层的封装，所以我们没有见过 `int 0x80`、`syscall`。
- 当操作系统发生缺页中断，处理内存碎片，除零野指针错误都会转化为 `CPU` 内部软中断，走中断处理流程，在系统调用函数指针表中都有对应的处理方法。
- 操作系统就是躺在中断处理例程上的代码块。操作系统就是基于各种中断运行的!!!
- `CPU` 内部的软中断，`int 0x80`、`syscall`，我们叫做陷阱。`CPU` 内部的软中断，除零、野指针，我们叫做异常

再谈用户态内核态

每一个进程的虚拟地址空间，有一个 `GB` 的内存是映射到操作系统上的，另外 `0-3GB` 是进程自己的地址空间。这意味着 `CPU` 在运行任意一个进程的时候，都可以找到同一个操作系统。操作系统的系统调用的执行，也是在进程的地址空间中进行的。

所谓内核态，就是进程在执行[3, 4]GB时所处的状态，用户态就是执行[0, 3]GB时处的状态。

那么当前进程是如何区分CPU执行的是内核态还是用户态呢？用户态和内核态之间是如何切换的呢？

CPU指令集：是CPU实现指挥硬件的媒介，每一条汇编指令都对应了一个指令，很多的汇编代码一起构成指令集。指令集直接涉及到硬件，是不可能让你直接接触到的，所以操作系统内核直接屏蔽掉了开发人员对硬件的直接操作。所以CPU的指令集也是有权限分级的哪些操作在当前权限可以做，哪些不能做。以intel CPU为例。将权限化为4个级别：

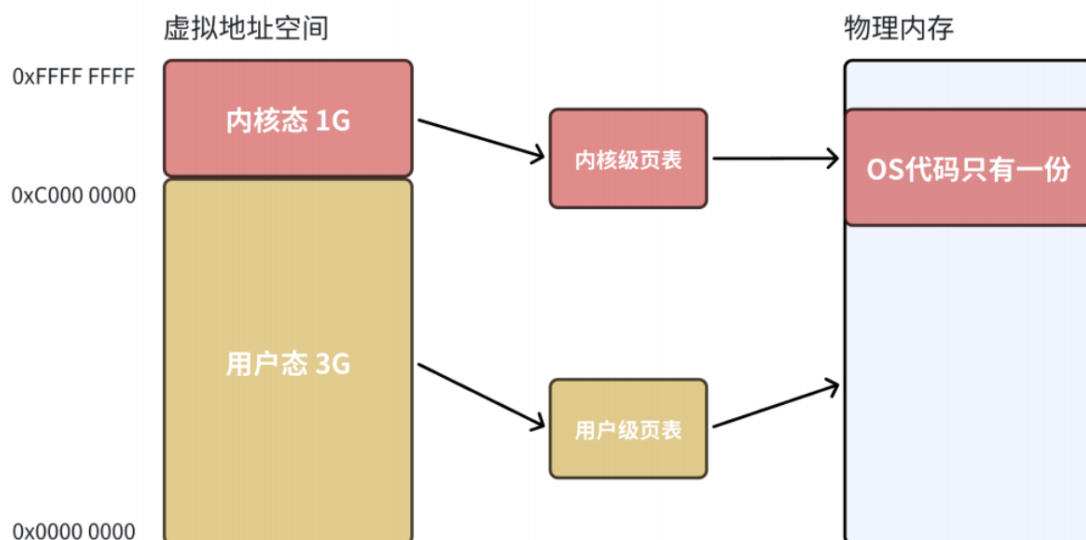
- ring0：权限最高，可以使用所有CPU指令集
- ring1：
- ring2：
- ring3：权限最低，仅能使用常规CPU指令集

CPU内部有一个字段，表示当前特权级别，那么用户态对应的就是3，内核态就是0。所以从用户态转变到内核态的本质其实就是将这个字段从3改为0即可。

当我们进行系统调用时，操作系统提供了中断 `int 0x80` 或者 `syscall` 主动进入内核，这是用户程序发起的调用访问内核代码的唯一方式。内核收到中断指令后查询中断向量表，随后进入内核态。

缺页异常和错误也是同理，CPU收到中断号后暂停当前执行进程，保存上下文，转而进入内核态处理缺页异常，处理错误。

在上面的过程中CPU从用户态转到内核态不但要进行提权，而且要切换对应的栈空间，[0-3GB]--->[3-4GB]，这个过程也会消耗一定的资源。



可重入函数

main 函数调用 insert 函数向一个链表 head 中插入节点 node1,插入操作分为两步,刚做完第一步的时候,因为硬件中断使进程切换到内核,再次回用户态之前检查到有信号待处理,于是切换到 sighandler 函数,sighandler 也调用insert函数向同一个链表 head 中插入节点 node2,插入操作两步都做完之后从 sighandler 返回内核态,再次回到用户态就从 main 函数调用的 insert 函数中继续往下执行,先前做第一步之后被打断,现在继续做完第二步。结果是,main 函数和 sighandler 先后 向链表中插入两个节点,而最后只有一个节点真正插入链表中了。

像上例这样,insert 函数被不同的控制流程调用,有可能在第一次调用还没返回时就再次进入该函数,这称为重入,insert 函数访问一个全局链表,有可能因为重入而造成错乱,像这样的函数称为 不可重入函数,反之,如果一个函数只访问自己的局部变量或参数,则称为可重 Reentrant 函数。想一下,为什么两个不同的控制流程调用同一个函数,访问它的同一个局部变量或参数就不会造成错乱?

如果一个函数符合以下条件之一则是不可重入的:

- 调用了malloc或free,因为malloc也是用全局链表来管理堆的。
- 调用了标准I/O库函数。标准I/O库的很多实现都以不可重入的方式使用全局数据结构。

volatile

volatile 作用：保持内存的可见性，告知编译器，被该关键字修饰的变量，不允许被优化，对该变量的任何操作，都必须在真实的内存中进行操作

volatile 是编程中的一个关键字，主要用于告知编译器或运行时环境，某个变量可能会被意外修改，从而避免对其进行优化或假设其值不变。它在多线程、硬件寄存器访问等场景中尤为重要。以下是其主要用途和示例：

1. 防止编译器优化

编译器在优化代码时，可能会将频繁访问的变量缓存到寄存器中以提高效率。但若该变量被其他线程或外部硬件修改，这种缓存会导致程序读取到**过时的值**。volatile 强制每次访问变量时都从内存中读取最新值。

示例：

```
1 volatile bool flag = false;
2
3 // 线程1
4 void wait_for_flag() {
5     while (!flag) { // 如果不加 volatile，编译器可能优化为只读取一次 flag
6         // 等待
7     }
8 }
9
10 // 线程2
11 void set_flag() {
12     flag = true; // 外部修改 flag
13 }
```

2. 保证可见性（多线程环境）

在支持多线程的语言（如 C++）中，volatile 确保一个线程对变量的修改能立即被其他线程看到（即**内存可见性**）。但需注意：

- **不保证原子性**：例如 volatile int i = 0; i++ 在多线程下仍可能出错（i++ 是非原子操作）。
- 替代方案：对于复合操作，需用 synchronized、Lock 或原子类（如 AtomicInteger）。

3. 访问硬件寄存器

在嵌入式或系统编程中，硬件寄存器（如状态寄存器、内存映射 I/O）的值可能被硬件自动改变。使用 volatile 可确保程序每次访问寄存器时都读取实际值。

示例（C/C++）：

```
1 volatile uint32_t *hardware_reg = (volatile uint32_t *)0x1234;
2
3 void read_register() {
4     uint32_t value = *hardware_reg; // 每次读取真实的硬件寄存器值
5 }
```

注意事项

- **不适用于复杂操作：** `volatile` 无法保证复合操作（如 `i++`）的原子性。
- **与锁的区别：** 锁（如 `synchronized`）既保证原子性又保证可见性，而 `volatile` 仅保证可见性。
- **语言差异：** C/C++ 的 `volatile` 仅防止编译器优化，Java 的 `volatile` 还包含内存屏障语义。

总结

使用 `volatile` 的场景：

1. 变量被多个线程共享且无需原子性保证。
2. 变量可能被异步修改（如中断服务程序、硬件寄存器）。
3. 需要快速、轻量级的可见性控制（如状态标志位）。

在更复杂的并发场景中，需结合锁、原子类或并发容器来确保线程安全。